

CSC 4520/6520

Design & Analysis of Algorithms

CHAPTER 4: DECREASE-AND-CONQUER

(GENERATING COMBINATORIAL OBJECTS)

Abdullah Bal, PhD

Objectives



Recap

Exhaustive search
Travelling salesman problem
Knapsack problem
Assignment problem
Graph Traversal
Decrease-and-Conquer
Insertion Sort
Topological Sorting



Generating
Permutations



Johnson
Trotter
Algorithm



Generating
Subsets

Algorithms for Generating Combinatorial Objects

- The most important types of **combinatorial** objects are
 - **permutations**
 - **combinations**
 - **subsets** of a given set
- They typically arise in problems that require a **consideration of different choices**.
- The number of combinatorial objects typically **grows exponentially** or even faster as a function of the problem size.
- Our primary interest here lies in algorithms for **generating combinatorial** objects, not just in counting them.

1-Generating Permutations: Minimal-change

- If $n = 1$ return 1;
- Otherwise, generate recursively the list of all permutations of
1 2... $n-1$
- Then insert n into each of those permutations by starting with inserting n into
1 2... $n-1$ by moving right to left.

1-Generating Permutations: Minimal-change

Example: $n=3$

start	1		
insert 2 into 1 right to left	12	21	
insert 3 into 12 right to left	123	132	312
insert 3 into 21 right to left	213	231	321

2-Generating Permutations: Johnson-Trotter Algorithm

- It is possible to get the same ordering of permutations of n elements without explicitly generating permutations for smaller values of n .
- It can be done by associating a direction with each element k in a permutation.
- We indicate such a direction by a small arrow written above the element in question, e.g.,

$$\begin{array}{cccc} \rightarrow & \leftarrow & \rightarrow & \leftarrow \\ 3 & 2 & 4 & 1 \end{array}$$

- The element k is said to be **mobile** in such an arrow-marked permutation if its arrow points to a smaller number adjacent to it.
- 3 and 4 are mobile while 2 and 1 are not.
- Using the notion of a mobile element, we can give the following description of the **Johnson-Trotter algorithm** for generating permutations.

2- Johnson-Trotter Algorithm (Cont.)

ALGORITHM *JohnsonTrotter*(n)

//Implements Johnson-Trotter algorithm for generating permutations

//Input: A positive integer n

//Output: A list of all permutations of $\{1, \dots, n\}$

initialize the first permutation with $\overleftarrow{1} \overleftarrow{2} \dots \overleftarrow{n}$

while the last permutation has a mobile element **do**

 find its largest mobile element k

 swap k with the adjacent element k 's arrow points to

 reverse the direction of all the elements that are larger than k

 add the new permutation to the list

2- Johnson-Trotter Algorithm (Cont.)

- An application of this algorithm for $n = 3$, with the largest mobile element shown in bold:

$\overleftarrow{1} \overleftarrow{2} \overleftarrow{\mathbf{3}} \quad \overleftarrow{1} \overleftarrow{\mathbf{3}} \overleftarrow{2} \quad \overleftarrow{3} \overleftarrow{1} \overleftarrow{\mathbf{2}} \quad \overrightarrow{\mathbf{3}} \overleftarrow{2} \overleftarrow{1} \quad \overleftarrow{2} \overrightarrow{\mathbf{3}} \overleftarrow{1} \quad \overleftarrow{2} \overleftarrow{1} \overrightarrow{3}.$

2- Johnson-Trotter Algorithm (Cont.)

- This algorithm is **one of the most efficient for generating** permutations
- It can be implemented to run in time proportional to the number of permutations, i.e., in $\Theta(n!)$.
- It is **horribly slow** for all but very small values of n ;
- This is not the algorithm's “fault” but rather the fault of the problem
- It simply asks to generate too many items.

Generating Subsets

- The knapsack problem, which asks to find **the most valuable subset of items that fits a knapsack** of a given capacity.
- The exhaustive-search approach to solving this problem discussed there was based on generating all subsets of a given set of items.
- We discuss algorithms for generating all 2^n subsets of an abstract set

$$A = \{a_1, \dots, a_n\}.$$

- Mathematicians call the set of all subsets of a set its **power set**.

1- Generating Subsets: Decrease-by-one

- The decrease-by-one idea is immediately applicable to this problem.
- All subsets of $A = \{a_1, \dots, a_n\}$ can be divided into two groups: those that do not contain a_n and those that do.
- The former group is nothing but all the subsets of $\{a_1, \dots, a_{n-1}\}$, while each and every element of the latter can be obtained by adding a_n to a subset of $\{a_1, \dots, a_{n-1}\}$.
- Thus, once we have a list of all subsets of $\{a_1, \dots, a_{n-1}\}$, we can get all the subsets of $\{a_1, \dots, a_n\}$ by adding to the list all its elements with a_n put into each of them.

1- Generating Subsets: Decrease-by-one

- An application of this algorithm (bottom-up) to generate all subsets of $\{a_1, a_2, a_3\}$ is illustrated as follow:

n		subsets							
0	$\emptyset$								
1	$\emptyset$	$\{a_1\}$							
2	$\emptyset$	$\{a_1\}$	$\{a_2\}$	$\{a_1, a_2\}$					
3	$\emptyset$	$\{a_1\}$	$\{a_2\}$	$\{a_1, a_2\}$	$\{a_3\}$	$\{a_1, a_3\}$	$\{a_2, a_3\}$	$\{a_1, a_2, a_3\}$	

2- Generating Subsets: Bit Strings

- A convenient way of solving the problem directly is based on a **one-to-one correspondence between all 2^n subsets** of an n element set $A = \{a_1, \dots, a_n\}$ and all 2^n bit strings $b_1, \dots, b_n$ of length n .
- The easiest way to establish such a correspondence is to assign to a subset the bit string in which $b_i = 1$ if a_i belongs to the subset and $b_i = 0$ if a_i does not belong to it.

2- Generating Subsets: Bit Strings

- For example, the bit string 000 will correspond to the empty subset of a three-element set, 111 will correspond to the set itself, i.e., $\{a_1, a_2, a_3\}$, and 110 will represent $\{a_1, a_2\}$.
- For the case of $n = 3$, we obtain

bit strings	000	001	010	011	100	101	110	111
subsets	$\emptyset$	$\{a_3\}$	$\{a_2\}$	$\{a_2, a_3\}$	$\{a_1\}$	$\{a_1, a_3\}$	$\{a_1, a_2\}$	$\{a_1, a_2, a_3\}$

Summary

- Generating Permutations
 - Minimal-change
 - Johnson Trotter Algorithm (Mobile Numbers)
- Generating Subsets
 - Decrease-by-one
 - Bit Strings

Supportive Materials

- *Introduction to The Design and Analysis of Algorithms (3rd Edition)* by Anany Levitin, Pearson.
- *Introduction to Algorithms*, by T.Cormen, C. Leiserson, R.Rivest and C.Stein, MIT Press, 3rd Edition.